

Analisi Tecnica dell'Handshake TCP

Laboratorio Cisco CyberOps: Analisi TCP

L'obiettivo di questo esercizio è analizzare il funzionamento dell'**handshake a tre vie** del protocollo TCP. Utilizzerai una topologia simulata in **Mininet** per generare traffico tra un client (H1) e un server web (H4).

Attraverso l'uso combinato di **tcpdump** per la cattura e **Wireshark** per l'ispezione dei pacchetti, imparerai a:

- Isolare i flag di controllo (**SYN, ACK**).
- Identificare le porte sorgente e destinazione.
- Verificare la corretta instaurazione di una sessione affidabile in una rete di produzione.

Dall'indagine effettuata, possiamo identificare chiaramente i tre passaggi fondamentali dell'handshake TCP tra l'host H1 (10.0.0.11) e il server H4 (172.16.0.40):

SYN (Pacchetto n. 10): L'host 10.0.0.11 invia un pacchetto con flag **[SYN]** al server sulla porta **80** (HTTP). Il numero di sequenza relativo è **0**.

```
10.0.0.11 → 172.16.0.40 [SYN] Seq=0 Win=0 Len=0 MSS=1460 SACK_PERM TSval=3335597126 TSecr=0 WS=512
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 3102866572
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment number (raw): 0
1010 ... = Header Length: 40 bytes (10)
  Flags: 0x002 (SYN)
    000. .... = Reserved: Not set
    ...0 .... = Accurate ECN: Not set
    .... 0... = Congestion Window Reduced: Not set
    .... .0.. = ECN-Echo: Not set
    .... ..0. = Urgent: Not set
    .... ...0 = Acknowledgment: Not set
    .... .... 0... = Push: Not set
    .... .... .0.. = Reset: Not set
    .... .... ..1. = Syn: Set
  [Expert Info (Chat/Sequence): Connection establish request (SYN): server port 80]
  [Connection establish request (SYN): server port 80]
  [Severity level: Chat]
  [Group: Sequence]
  .... .... ...0 = Fin: Not set
  [TCP Flags: .....S.]
Window: 42340
```

Porta TCP di origine: **56666** (nello screenshot) o **58716** (nel testo del lab). È classificata come una **porta effimera** o dinamica (assegnata dal sistema operativo del client).

Porta TCP di destinazione: **80**. È classificata come una **porta ben nota** (Well-Known Port) associata al protocollo HTTP.

Flag impostato: Solo il flag **SYN** (Synchronize).

Valore numero sequenza relativo: È impostato a **0**.

SYN, ACK (Pacchetto n. 11): Il server **172.16.0.40** risponde con i flag **[SYN, ACK]** impostati. Il numero di acknowledgment relativo è **1**, indicando che il server è pronto a ricevere il prossimo byte.

11	1.844951	172.16.0.40	10.0.0.11	TCP	66.00 - 56666 [SYN, ACK] Seq=1392 Min=43520 Len=0 TSval=1823875731 TSecr=3335597126 Win=512
14	1.844957	172.16.0.40	10.0.0.11	TCP	66.00 - 56666 [ACK] Seq=1 Ack=332 Min=43520 Len=0 TSval=1823875731 TSecr=3335597126
15	1.841359	172.16.0.40	10.0.0.11	TCP	384.00 - 56666 [FIN, ACK] Seq=1 Ack=332 Min=43520 Len=238 TSval=1823875731 TSecr=3335597126 [TCP PDU reassembled in 17]
17	1.841364	172.16.0.40	10.0.0.11	HTTP	481 HTTP/1.1 200 OK (text/html)

Sequence Number: 0 (relative sequence number)

Sequence Number (raw): 2591602311

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 332 (relative ack number)

Acknowledgment number (raw): 318286573

1618 ... = Header Length: 48 bytes (18)

Flags: 0x018 [SYN, ACK]

0000 ... = Reserved: Not set

...0 ... = Accurate ECN: Not set

...0 ... = Congestion Window Reduced: Not set

...0 ... = ECH-echo: Not set

...0 ... = Urgent: Not set

...1 ... = Acknowledgment: Set

...0 ... = Push: Not set

...0 ... = Reset: Not set

...0 ... = SYN: Not set

[Expert Info (Chat/Sequence): Connection establish acknowledge (SYN+ACK): server po

[Connection establish acknowledge (SYN+ACK): server port 80]

[Severity Level: Chat]

[Group: Sequence]

...0 ... = Fin: Not set

[TCP Flags:A..S]

Window: 43448

[Calculated window size: 43448]

Checksum: 0xb671 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

Options: (20 bytes). Maximum segment size, SACK permitted, Timestamps, No-Operation (NOP)

- **Porte:** L'origine è **80** (Server) e la destinazione è la porta effimera del client.
- **Flag impostati:** Sono impostati sia **SYN** che **ACK**.
- **Numeri relativi:** Il numero di sequenza è **0** e il numero di acknowledgment è **1**.

ACK (Pacchetto n. 12): L'host H1 risponde con il flag **[ACK]** per confermare la ricezione del pacchetto del server. A questo punto, la sessione è stabilita.

12	1.844957	172.16.0.40	10.0.0.11	TCP	66.00 - 56666 [ACK] Seq=1 Ack=332 Min=43520 Len=0 TSval=1823875731 TSecr=3335597126
15	1.841359	172.16.0.40	10.0.0.11	TCP	384.00 - 56666 [FIN, ACK] Seq=1 Ack=332 Min=43520 Len=238 TSval=1823875731 TSecr=3335597126 [TCP PDU reassembled in 17]
17	1.841364	172.16.0.40	10.0.0.11	HTTP	481 HTTP/1.1 200 OK (text/html)

Sequence Number: 1 (relative sequence number)

Sequence Number (raw): 2591602312

[Next Sequence Number: 1 (relative sequence number)]

Acknowledgment Number: 332 (relative ack number)

Acknowledgment number (raw): 318286594

1088 ... = Header Length: 32 bytes (8)

Flags: 0x018 [ACK]

0000 ... = Reserved: Not set

...0 ... = Accurate ECN: Not set

...0 ... = Congestion Window Reduced: Not set

...0 ... = ECH-echo: Not set

...0 ... = Urgent: Not set

...1 ... = Acknowledgment: Set

...0 ... = Push: Not set

...0 ... = Reset: Not set

...0 ... = SYN: Not set

...0 ... = FIN: Not set

[TCP Flags:A....]

Window: 65

[Calculated window size: 43520]

[Window size scaling factor: 512]

Checksum: 0xb669 [unverified]

[Checksum Status: Unverified]

Urgent Pointer: 0

- **Porte:** L'origine è **80** (Server) e la destinazione è la porta effimera del client.
- **Flag impostati:** E' impostato solamente il flag di Acknowledgment.
- **Numeri relativi:** Il numero di sequenza è **1**

Filtri Wireshark utili:

- **tcp.flags.syn == 1 and tcp.flags.ack == 0:** Isola solo i pacchetti di richiesta connessione iniziale (utilissimo per rilevare scansioni SYN).
- **ip.src == 10.0.0.11:** Filtra il traffico proveniente esclusivamente da uno specifico host sorgente per monitorarne l'attività.
- **http.request.method == "GET":** Mostra solo le richieste di download di risorse web, filtrando le risposte e altri pacchetti TCP.

Altri usi in produzione:

- **Analisi di protocolli crittografati:** Caricando le chiavi private, Wireshark può essere usato per decifrare traffico TLS e ispezionare il contenuto delle comunicazioni sicure.
- **Baselines di rete:** Creare profili di traffico "normale" per poter identificare immediatamente scostamenti sospetti o picchi di carico insoliti.
- **Ottimizzazione delle prestazioni:** Valutazione della latenza di rete e del tempo di risposta dei server esaminando i timestamp dei pacchetti catturati.
- **Risoluzione dei problemi di connettività (Troubleshooting):** Analisi dei ritardi, delle ritrasmissioni dei pacchetti o dei reset delle connessioni per identificare guasti hardware o configurazioni errate.